A Fully Dynamic Reachability Algorithm for Directed Graphs with an Almost Linear Update Time

Liam Roditty, Uri Zwick

汪逸阳,2022.6.3…拖延到了2022.12.19

目录

- 介绍与一些前置知识
- 动机
- 算法：强联通分量关系的维护
- 算法：可达性树的维护
- 最终算法

Dynamic Reachability Problem

- 在有向图$G = G(V, E)$经历一系列的添加边和删除边（更新）的操作时，回答任意两点是否可达（查询）。

- 该文章提出了一个牺牲了一定查询时间效率来换取更低的更新开销的动态可达性算法。

  更新（均摊分析）：添加边$O(m + nlogn)$，删除边$O(m\alpha(m, n))$

  查询：最差情况下$O(n)$。（其中n为图中节点的个数，m为更新时的边数）

$Insert(E^v)$	–	Insert a set of edges E^v, all touching a given vertex v, into the graph.
$Delete(E')$	–	Delete all the edges of E' from the graph. The set E' is an arbitrary set of edges.
$Query(u, v)$	–	Check whether there is a directed path from u to v in the current graph.

Preliminaries

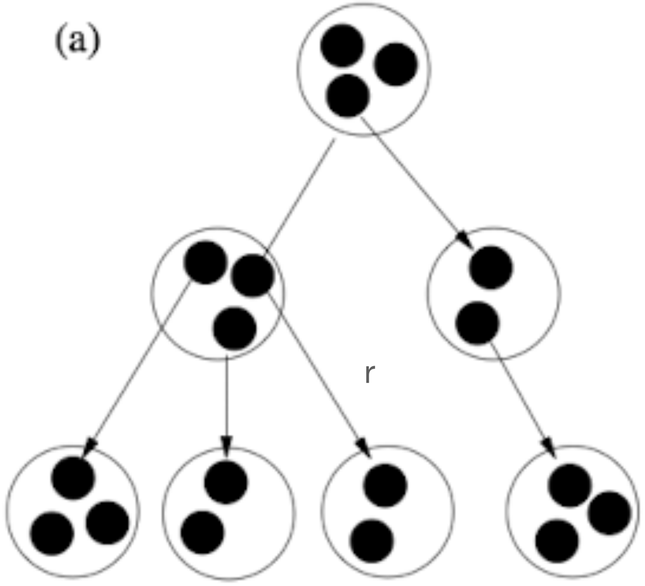
- 强连通：在有向图G中，如果两个顶点v_i, v_j间，有一条从v_i到v_j的有向路径，同时还有一条从v_j到v_i的有向路径，则称两个顶点强连通(strongly connected)。
- 强连通图：有向图G的任意两个顶点强连通。
- 强连通分量：极大的强连通子图。
- 有向无环图（DAG）：一个无法从某个顶点出发经过若干条边回到该点的有向图。

个人加上的motivation（1）

- 若u, v间可达，那么u到v的路径上的所有点w都满足，u可达w且w可达v。
- 那么如果能保证，在查询u, v间的可达性的时候，至少有一个在u到v路径上的节点w，为其记录下的可达性关系覆盖了整条可达性路径，那么只要检查到了该点w就能说明u, v是可达的，而若找不到该点则说明不可达。
- 如此以来，我们最多只需要检查n个点，且若检查一个点的时间代价是常数级别的，那么查询的代价为$O(n)$级别。
- 同时，由于只需要维护部分的可达性关系，故更新代价会相对下降。

个人加上的motivation（2）

- 一个点r到其他的点的可达性关系可以表示成一棵可达性树，我们可以记录下点r与其他店的可达性关系，从而能做到在常数时间内回答。
- 建立起一棵可达性树可以容易地在$O(m)$时间内完成。
- 但当发生删除边操作的时候，树中的节点（强联通分量）可能会分裂为多个分量，也可能会失去其连接在树上的树边，因此我们需要有个手段来重建该树。
- 首先我们要知道一个强联通分量会分裂成哪些分量、哪些边可能会成为新树边，这就引出了后续的强联通分量森林。



大圈：强联通分量（分量）
黑色圆：图节点

强联通分量森林

- 考虑一系列图（版本）$G_0, G_1, G_2, \ldots, G_t$，$t$为添加边操作的执行次数，$G_0$为空图。

  有：$E_0 \subseteq E_1 \subseteq E_2 \subseteq \ldots \subseteq E_t$

  因此：G_i的强联通分量为G_{i-1}的一个强联通分量或者是G_{i-1}的数个强联通分量的并。

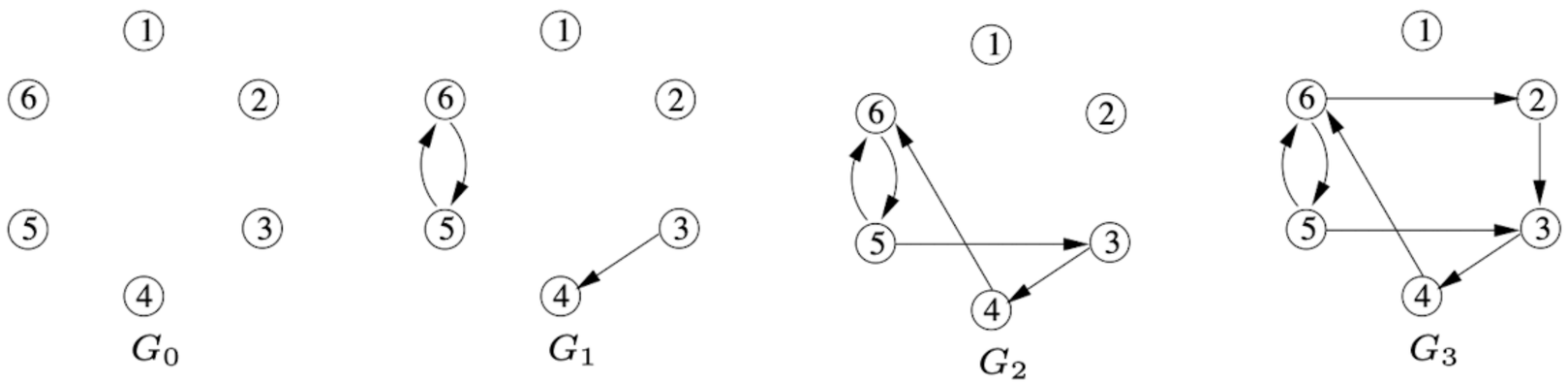
- 这种层级关系可以被表示为一个森林。

$\boldsymbol{Insert(E')}$: $t \leftarrow t+1$, $E_t \leftarrow E_{t-1} \cup E'$

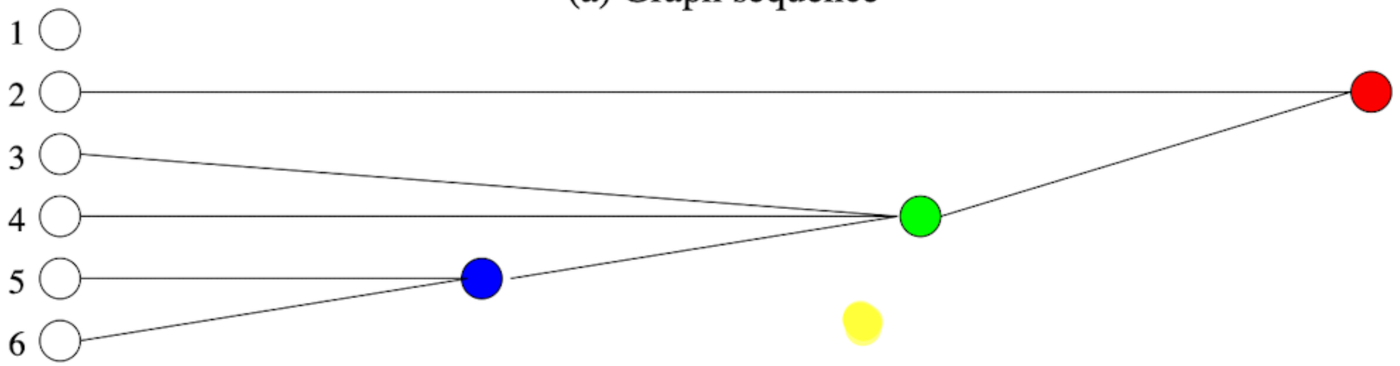
$\boldsymbol{Delete(E')}$: $E_i \leftarrow E_i - E'$, for $1 \leq i \leq t$

$\boldsymbol{Query(u, v, i)}$: Are u and v in the same component of the graph G_i?

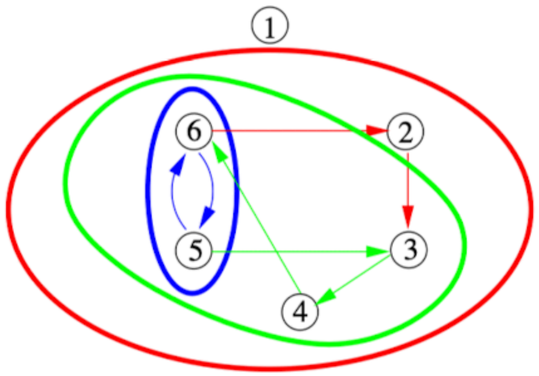
森林示意图



(a) Graph sequence



(b) Components forest



(c) Components hierarchy

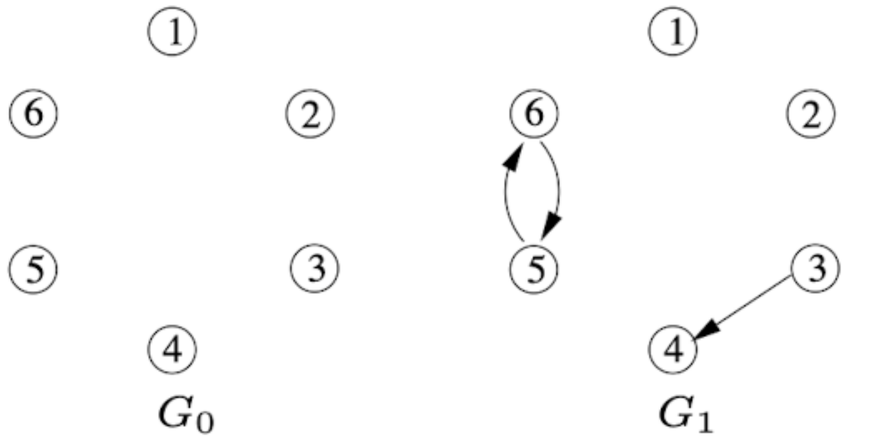
维护森林（1）

动态边划分图

- 动态边划分集合$H_1, \ldots, H_t, H_{t+1}$
- $E_t = \bigcup_{i=1}^{t+1} H_i$
- $H_i \cap H_j = \varnothing, i \neq j$

DEFINITION 2.1 (DYNAMIC EDGE PARTITIONING). *The dynamic edge set H_i of G_i is defined as follows:*

$$
\begin{aligned}
H_i &= \{ (u,v) \in E_i \mid Query(u,v,i) \wedge \\
&\qquad (\neg Query(u,v,i-1) \vee (u,v) \notin E_{i-1}) \} \\
H_{t+1} &= E_t \setminus \cup_{i=1}^{t} H_i
\end{aligned}
$$



- $t = 1$
- $H_1 = \{(5,6), (6,5)\}$
- $H_2 = \{(3,4)\}$

维护森林（2）

初始化

- 所需数据结构：

1.$parent$数组，$parent[v]$代表v在森林中的父节点。

2.$version$数组，$version[v]$代表强联通分量v首次出现的图版本号。

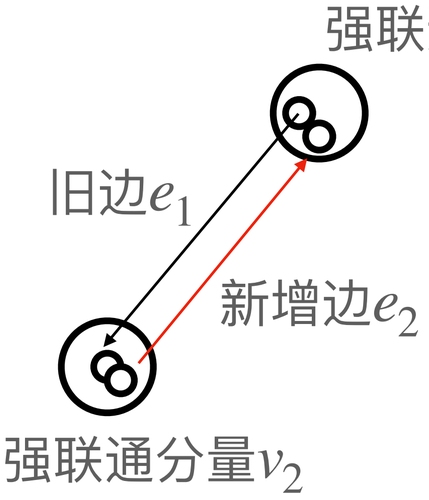
Init:

1. $t \leftarrow 0$
2. $H_1 \leftarrow \phi$
3. for each $v \in V$ do
4. $\quad parent[v] \leftarrow null$
5. $\quad version[v] \leftarrow 0$

维护森林（3）

时间复杂度：最差情况下为$O(m\alpha(m,n))$

增添边



$H_t = \{e_1\} \cup \{e_2\} = \{e_1, e_2\}$

$H' = \{e_1', e_2'\}$，将强联通分量视为一个节点。

$SCC(H')$返回$\{\{v_1, v_2\}\}$

发现新分量$\{v_1, v_2\}$，构造新的森林节点c，设置其版本为当前版本，并将v_1, v_2通过并查集union操作合并，并将它们的parent设置为新的分量c。

Insert(E'):

1. $t \leftarrow t+1$
2. $H_t \leftarrow H_t \cup E'$
3. $FindScc(H_t, t)$
4. $Shift(H_t, H_{t+1})$
5. $Pre\text{-}LCA(parent)$

FindScc(H, i):

1. $H' \leftarrow \{(Find(u), Find(v)) \mid (u,v) \in H\}$
2. $\mathcal{C} \leftarrow SCC(H')$
3. for each $C = \{w_1, w_2, \ldots, w_{|C|}\} \in \mathcal{C}$ do
4. if $|C| > 1$ then
5. $c \leftarrow NewNode$
6. $version[c] \leftarrow i$
7. for $j \leftarrow 1$ to $|C|$
8. if $j > 1$ then $Union(w_1, w_j)$
9. $parent[w_j] \leftarrow c$

Shift(H_1, H_2):

1. for each $(u,v) \in H_1$
2. if $Find(u) \neq Find(v)$ then
3. $H_1 \leftarrow H_1 \setminus \{(u,v)\}$
4. $H_2 \leftarrow H_2 \cup \{(u,v)\}$

SCC为寻找图中的强联通分量的算法，可参考算法导论22.5。

维护森林（4）

删除边

- 从第一个图版本开始，依次从动态边划分集合H_i中删去待删除边，重新自底向上构造新的森林。

- 均摊时间复杂度为：$O(m\alpha(m,n)+t)$。

$\boldsymbol{Delete(E')}$:

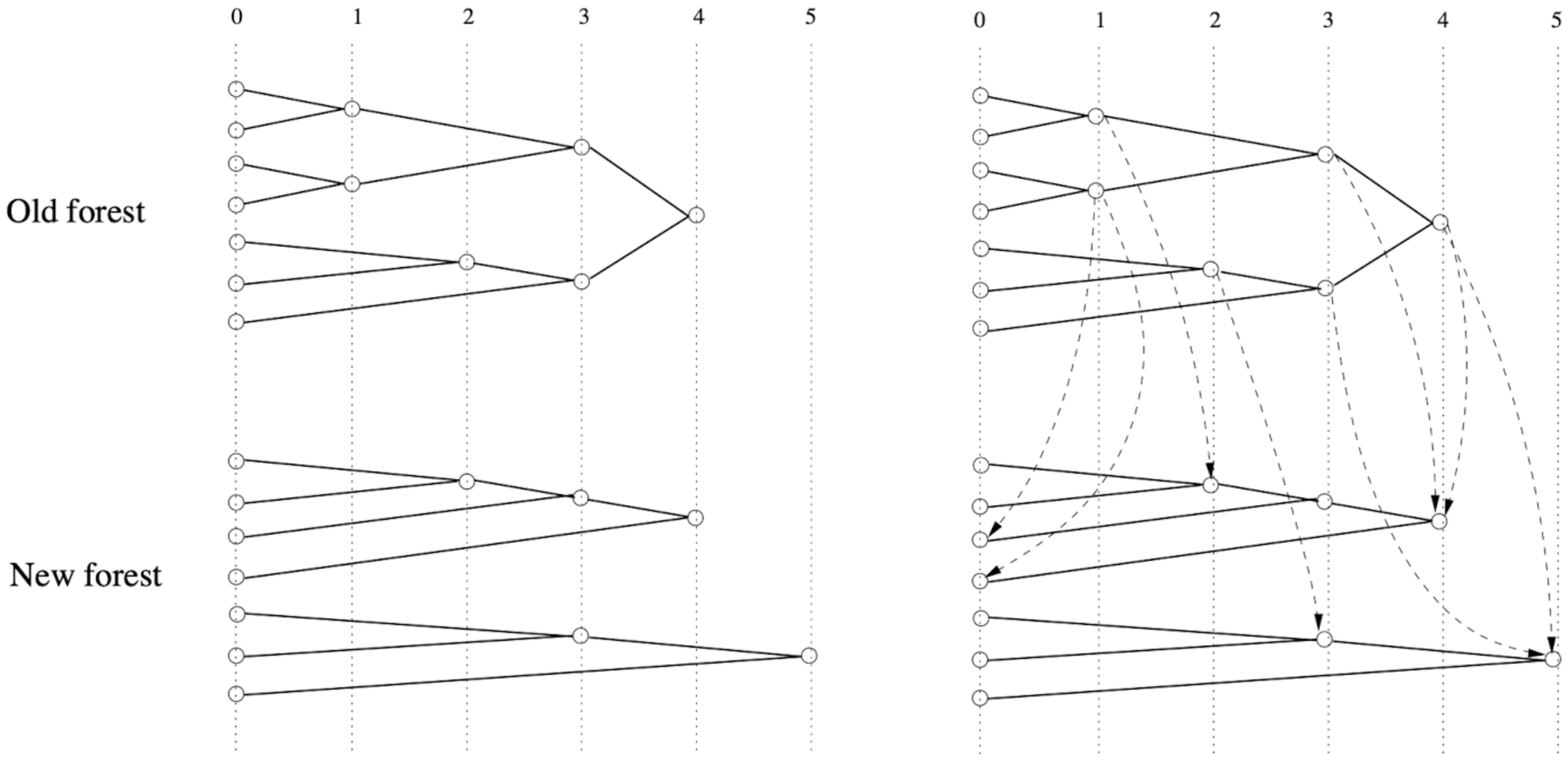
1. for each $v \in V$ do
2. $\quad parent[v] \leftarrow null$
3. for $i \leftarrow 1$ to t
4. $\quad H_i \leftarrow H_i \setminus E'$
5. $\quad FindScc(H_i, i)$
6. $\quad Shift(H_i, H_{i+1})$
7. $H_{t+1} \leftarrow H_{t+1} \setminus E'$
8. $Pre\text{-}LCA(parent)$

$\boldsymbol{FindScc(H, i)}$:

1. $H' \leftarrow \{(Find(u), Find(v)) \mid (u,v) \in H\}$
2. $\mathcal{C} \leftarrow SCC(H')$
3. for each $C = \{w_1, w_2, \ldots, w_{|C|}\} \in \mathcal{C}$ do
4. $\quad$ if $|C| > 1$ then
5. $\quad\quad c \leftarrow NewNode$
6. $\quad\quad version[c] \leftarrow i$
7. $\quad\quad$ for $j \leftarrow 1$ to $|C|$
8. $\quad\quad\quad$ if $j > 1$ then $Union(w_1, w_j)$
9. $\quad\quad\quad parent[w_j] \leftarrow c$

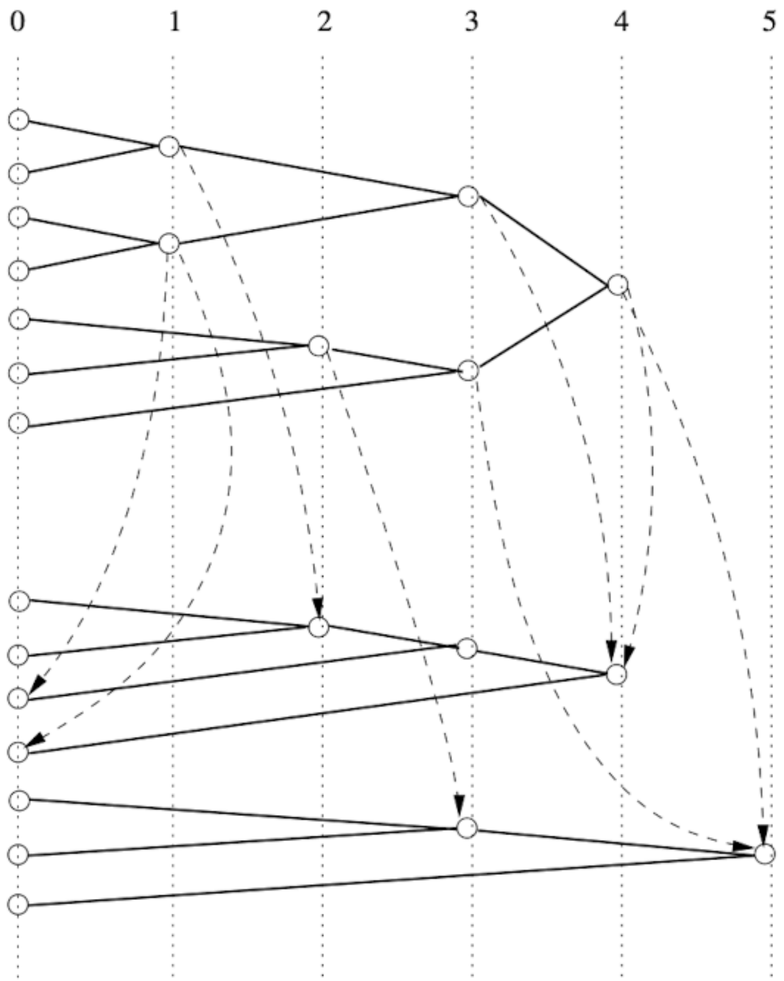
维护森林（5）

强联通分量的分解



维护森林（6）

强联通分量的分解



$Parts(v)$：返回v经过删除边后分解成的最大强联通分量集合。

$Parts(v)$:

1. $S \leftarrow \cup_{v' \in children[v]} Parts(v')$
2. $T \leftarrow \{\, parent[u] \mid u \in S \wedge parent[u] \neq null \,\}$
3. while $T \neq \phi$ do
4. $\quad u \leftarrow extract\text{-}min(T, version)$
5. $\quad$ if $children[u] \subseteq S$ then
6. $\quad\quad S \leftarrow S \cup \{u\} - children[u]$
7. $\quad\quad$ if $parent[u] \neq null$ then $T \leftarrow T \cup \{parent[u]\}$
8. return S

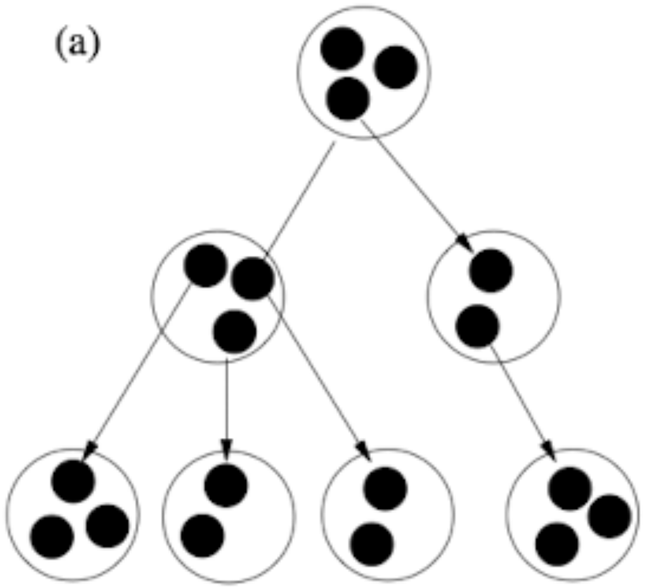
$Split(v, i)$：返回v分解成的最大强联通分量集合，且这些强联通分量都在版本i前出现

$Split(v, i)$:

1. $R \leftarrow Parts(v)$
2. while $\exists u \in R$ with $version[u] > i$ do
3. $\quad R \leftarrow R \cup children[u] - \{u\}$
4. return R

可达性树

- 一棵可达性树用来维护一个特定顶点r在图$G = G(V, E)$中经过一系列的删除边操作时的可达顶点集合。
- 树的节点是强联通分量，根节点为包含r的强联通分量。



维护可达性树（1）

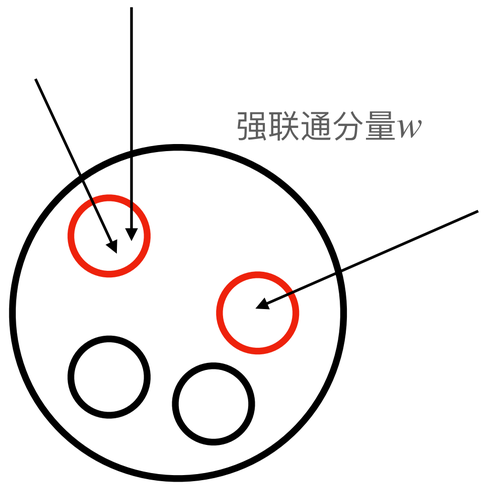
数据结构

- 对图中的每个分量w，维护一个双向链表$active[w]$，其中包含了w的所有活跃节点。

  一个节点v为活跃的当且仅当其存在未检查的且跨越分量的入边。

  初始时所有边均为未检查的；树边将被一直看作是未检查的边。

- 对每个活跃节点v，维护一个双向链表$in[v]$，其中包含了v的所有未检查且跨越分量的入边。



维护可达性树（2）

数据结构

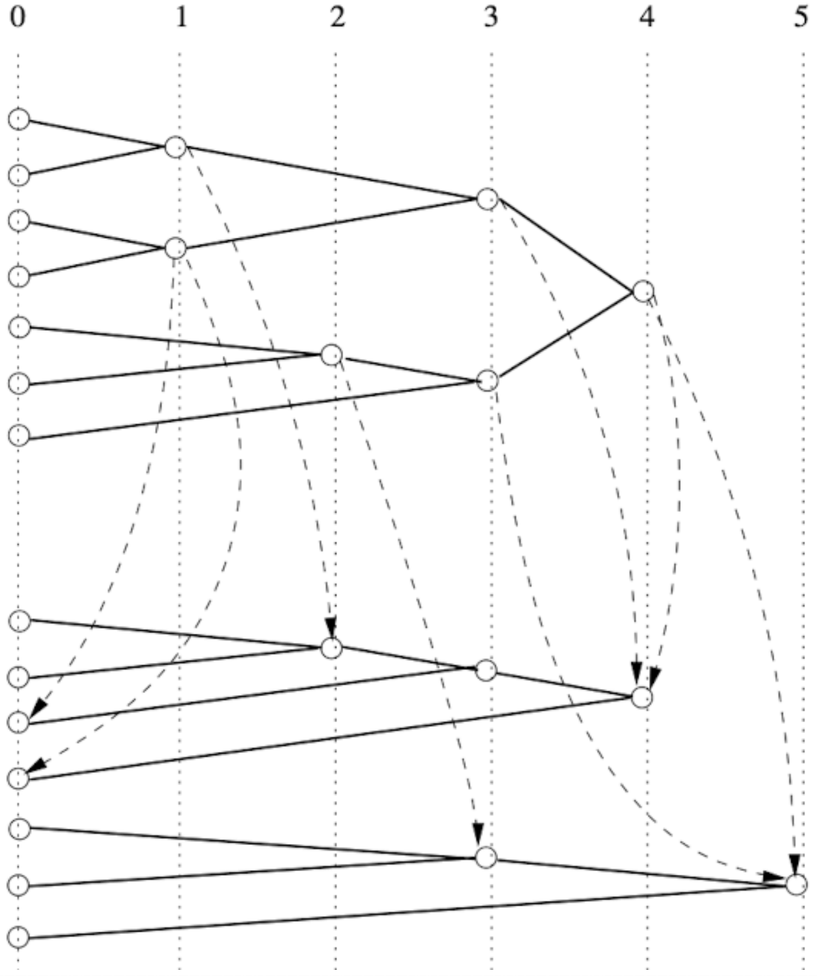
- 对图中的每个节点v均维护一个双向链表$out[v]$，其中包含v的所有出边。
- 维护一个数组c，$c[v]$表示当前包含v的强联通分量。
- 算法会维护一个不变性质：
  w为图G中的一个分量，v是$active[w]$中的首个节点，u是$in[v]$的首个节点，则(u, v)是将w连接在可达性树上的树边。

维护可达性树（3）

设置新分量的active链表

- 通过强联通分量森林，可以得到在该可达性树版本i下，删除前的分量w在边删除后分解成的最大分量$w_1, \ldots, w_k$。
- 维护森林中的删除操作中，从动态划分图H_i转移到H_{i+1}中的边即为在删除边操作后成为了跨分量边的边。
- 因此这些未检查的跨分量的边指向的节点为活跃的，可以分别设置active链表。

通过一个tricky的方法，一棵可达性树无论经历多少次删除的操作，构造active链表所用的时间都不会超过O(nlogn)



$\boldsymbol{Delete(E')}$:

1. for each $v \in V$ do
2. $\quad parent[v] \leftarrow null$
3. for $i \leftarrow 1$ to t
4. $\quad H_i \leftarrow H_i \setminus E'$
5. $\quad FindScc(H_i, i)$
6. $\quad Shift(H_i, H_{i+1})$
7. $H_{t+1} \leftarrow H_{t+1} \setminus E'$
8. $Pre\text{-}LCA(parent)$

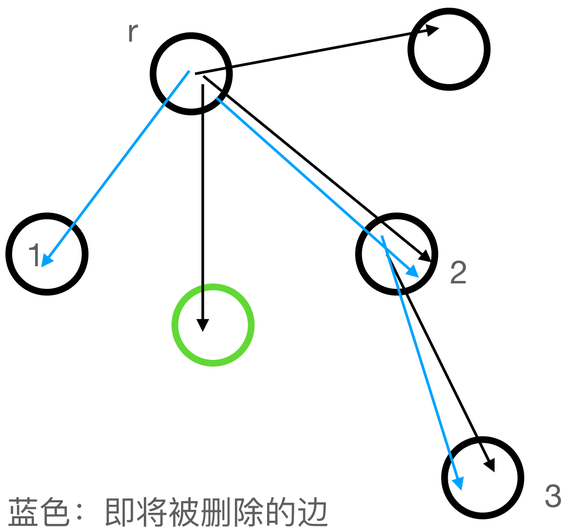
维护可达性树（4）

在边删除后重建树

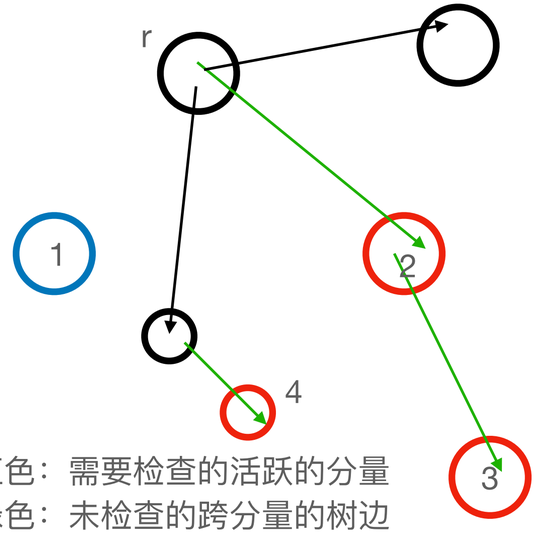
- W为需检查的分量的集合，其中的分量要么是原有的分量但丢失了原先将其连接在树上的树边（1、2、3），要么是新分解出的但没有原先的连接树边的分量（4）。

- 对$w \in W$，算法会遍历其活跃点v的入边(u, v)，若满足$active[c[u]] \neq \varnothing \vee c[u] = c[r]$，则尝试将$w$连接在$c[u]$上，并将$w$从$W$中移除；若不满足则该边称为检查过的边，从活跃点的入边集合中移除，若集合变为空则该点则变为不活跃，从$active[w]$中移除。

- 如果没有找到满足上述条件的边，则将w从W中移除，同时检查w中的所有跨分量的出边(u, v)，若该出边连接了一个尝试连接在w上的分量w'，那么则将w'重新加入W中。



蓝色：即将被删除的边
绿色：该分量内部有边将被删除

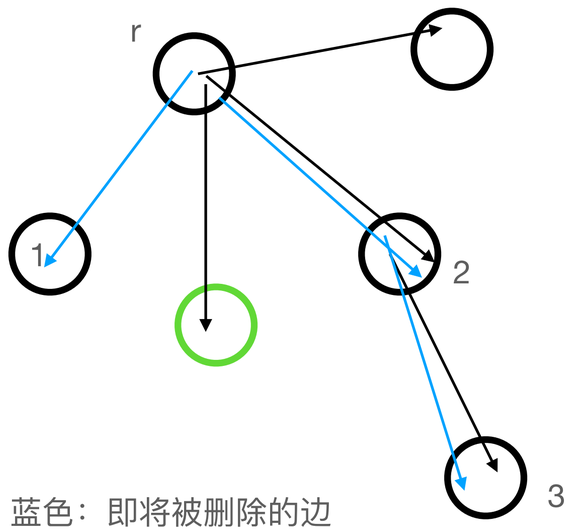


红色：需要检查的活跃的分量
绿色：未检查的跨分量的树边
蓝色：需要检查的非活跃分量
黑色：无需检查的分量

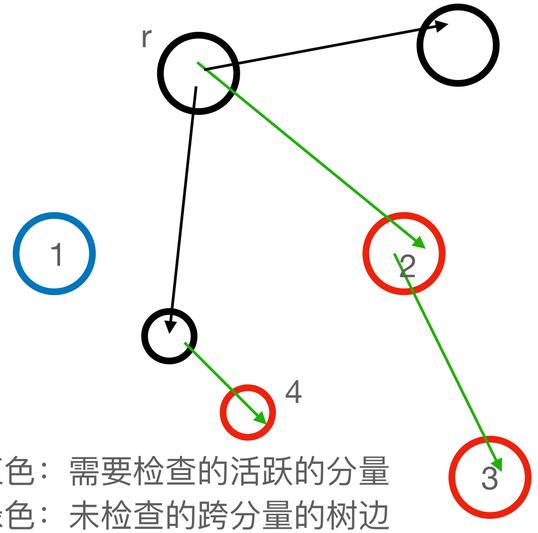
维护可达性树（5）

时间复杂度

- 维护active链表的时间$O(nlogn)$。
- 无论重建多少次树，一条边都最多只会被检查一次，因此无论重建多少次树，总共用的时间都是$O(m)$，其中m为该树被创建时图的边数。
- 因此维护可达性树的均摊时间复杂度为$O(nlogn+m)$。



蓝色：即将被删除的边
绿色：该分量内部有边将被删除



红色：需要检查的活跃的分量
绿色：未检查的跨分量的树边
蓝色：需要检查的非活跃分量
黑色：无需检查的分量

动态可达性算法（1）

- 每执行一次以点v为中心的增添边操作的时候，创造一对树$T_{in}[v], T_{out}[v]$，分别代表能到达v的分量和能从v到达的分量集合。（如果这对树已经存在了就直接覆盖，故可达性树的数量级为$O(n)$）
- 每次删除操作的时候，对所有的可达性树都进行重建操作。
- 查询操作只需要找到一个点w使得$u \in T_{in}[w] \wedge v \in T_{out}[W]$，该操作只需要遍历所有的树查询即可，因此时间复杂度为$O(n)$。 如此的空间复杂度应该要达到$O(n^2)$

$Insert(E^v)$	– Insert a set of edges E^v, all touching a given vertex v, into the graph.
$Delete(E')$	– Delete all the edges of E' from the graph. The set E' is an arbitrary set of edges.
$Query(u, v)$	– Check whether there is a directed path from u to v in the current graph.

动态可达性算法（2）

正确性

$Insert(E^v)$	–	Insert a set of edges E^v, all touching a given vertex v, into the graph.
$Delete(E')$	–	Delete all the edges of E' from the graph. The set E' is an arbitrary set of edges.
$Query(u, v)$	–	Check whether there is a directed path from u to v in the current graph.

- 查询操作只需要找到一个点w使得$u \in T_{in}[w] \wedge v \in T_{out}[W]$，该操作只需要遍历所有的树查询即可，因此时间复杂度为$O(n)$。
- 如果确实存在该点w，那么显然存在一条$u \to w \to v$的路径。
- 如果当前图中存在了一条u到v的路径，不妨设w为路径上的最后（时间意义上）一个充当为增添边操作的中心点，那么路径上的所有边都会在其可达性树$T_{in}[w], T_{out}[w]$上体现出来，因此该点w一定会被找到。

动态可达性算法（2）

时间复杂度

- 添加边时需要：新建可达性树：$O(m)$，该可达性树未来的所有维护开销（来自删除边操作）：$O(nlogn + m)$，维护强联通分量森林的开销$O(m\alpha(m,n))$，因此添加边的均摊时间复杂度为$O(m + nlogn)$。
- 删除边时需要：维护强联通分量森林$O(m\alpha(m,n))$，维护所有可达性树（该开销已被添加边操作均摊），因此均摊时间复杂度为$O(m\alpha(m,n))$。
- 查询可达性则为$O(n)$。